Data Structures
Topic: Binary Search Trees

Q1. Write C++ function for node deletion in BST.

Q2. Write C++ function to search a given value

Q3. Write a C++ function to search the given value if it found then return the node itself along with the parent.

Q4. Write C++ function to count all the values above the given value v in a BST

Q5. Write C++ function to find the minimum value in a BST.

Q6. Write C++ function to make use of queue to give breadth first traversal of a given BST.

Q7. Write C++ function to make use of queue to give breadth first Insertion of a given BST.

Q8. Write a C++ program that creates a sorted Linked List using binary search tree. (Note: Make use of the Linked List class that you have already created in lab)

Q9. Write a C++ program to display all terminal nodes of a binary search tree and then delete them.

Q10. Write an iterative method for traversing a binary search tree in post order. The node class should contain another data member visited that will be set to -1 when no children visited and will reset to 1 or 2 when both children will be visited.

```
class node
{
    int data;
    node left;
    node right;
    int visited;
    node ( int data)
    {

            this.data=data;
            visited=-1; // initially when no children is visited

    }
 }
```

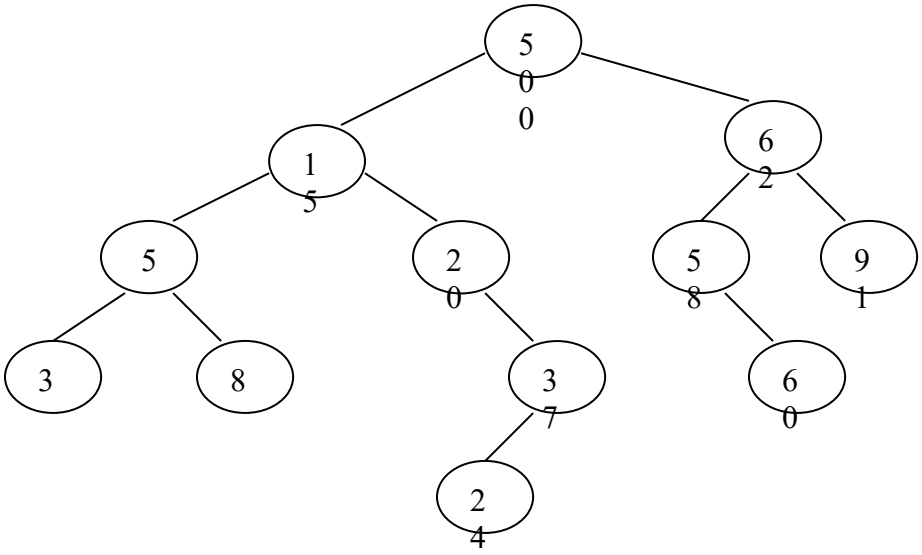
Q11.

a) Write a c++ function to print out all leaf node of a binary search tree?

b) Write code segment to search smallest and largest element in a BST and then swap both the nodes (instead of data) with each other. Does resulting tree would be A BST or not explain?

c) State whether the following tree is balanced or not. If not, balance the tree and perform the following operations on this AVL tree. Show the tree and clearly state the rotations at each step.

Insert 7, 59, then delete 60, 50, and then insert 6.



Q12.

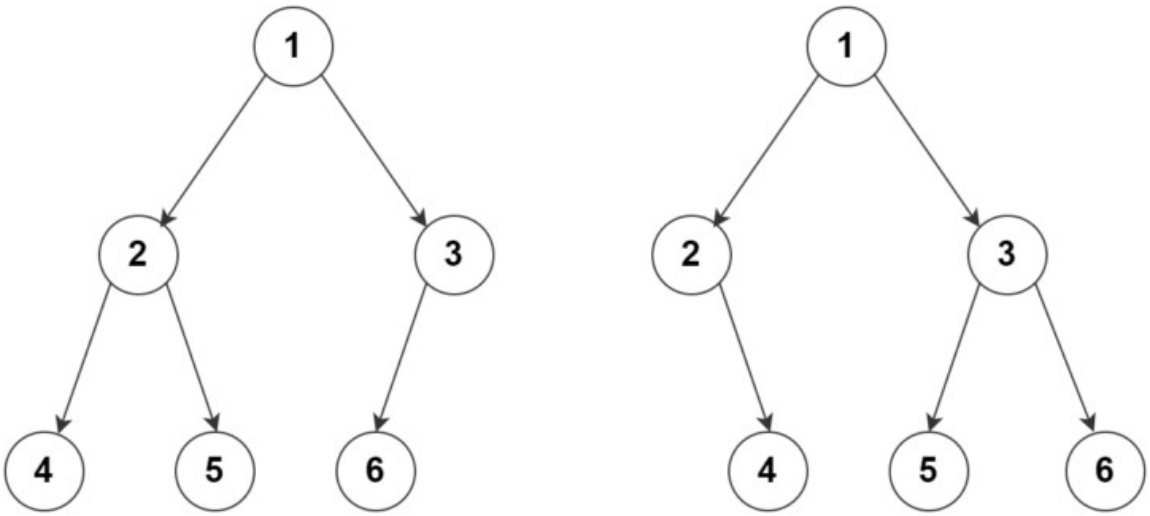
a) Show the result of inserting 11, 22, 3, 4, 16, 55, 7, 10, 33, 6, 14, 1,18, 33, and 12, one at a time, in an initially empty max heap, showing the array representation as well as tree side by side.

b) Write the priority queue deletion function code using max heap in c++.

Q13.

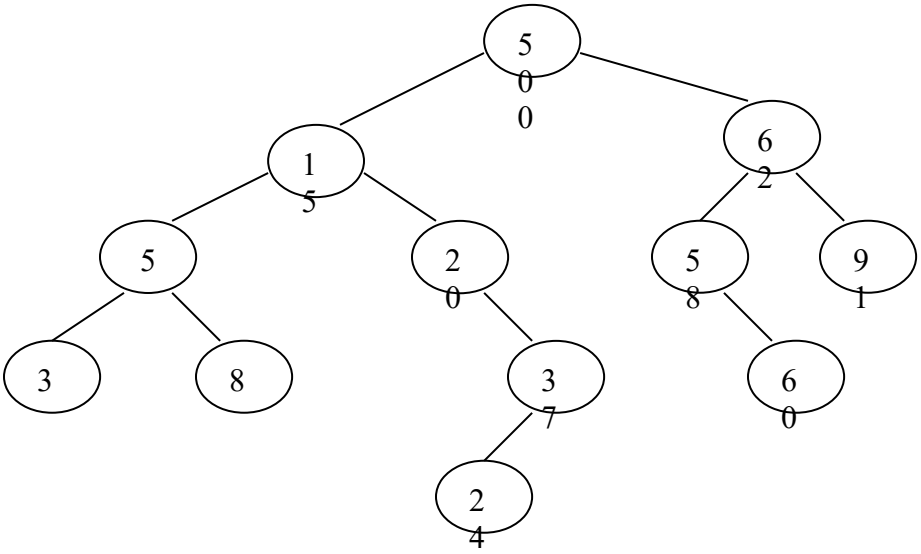
Given two binary trees, check whether the leaf traversals of both trees are the same or not.

For example, the leaf traversals of the following binary trees are `4, 5, 6`:



A simple solution is to traverse the first tree using **inorder traversal** and store each encountered leaf in an array. Repeat the same for the second tree. Then the problem reduces to **comparing two arrays for equality**. The time complexity of this approach is `O(m + n)`, and the additional space used is `O(m + n)`. Here `m` is the total number of nodes in the first tree, and `n` is the total number of nodes in the second tree.

Q14. Write a C++ code to search and print all the paths from the root to each of its leaf. Output for the sample tree given below is:
path 1: 50 ->15 ->5->3
path 2: 50 ->15->5->8
path 3: 50->15->20->37->24
Path 4: 50-> 62->58->60
Path 5: 50 ->62->91



Q15. Find the level of the node in a tree. (Note: rote is at level 0).
Q16. Print all the nodes at kth level of the tree.
Q17. Find height of a particular node in a tree. (Note: leaf has height 0).